Programming Assignment Report

Design and Analysis of Algorithms

Name (as in ERP): Muhammed Nadir Misbahi

Registration Number (in full):L2411021061698

Problem Name: Marc's Cakewalk

Problem Statement: Marc loves cupcakes, but he also likes to stay fit. Each cupcake has a calorie count, and Marc can walk a distance to expend those calories. If Marc has eaten j cupcakes so far, after eating a cupcake with c calories he must walk *at least* 2j*c miles to maintain his weight.

Example.

$calorie = [5, 10, 7]$

If he eats the cupcakes in the order shown, the miles he will need to walk are $(2^0 \times 5) + (2^1 \times 10) + (2^2 \times 7) = 5 + 20 + 28 = 53$. This is not the minimum, though, so we need to test other orders of consumption. In this case, our minimum miles is calculated as $(2^0 \times 10) + (2^1 \times 7) + (2^2 \times 5) = 10 + 14 + 20 = 44$.

Given the individual calorie counts for each of the cupcakes, determine the minimum number of miles Marc must walk to maintain his weight. Note that he can eat the cupcakes in any order.

Function Description

Complete the *marcsCakewalk* function in the editor below.

marcsCakewalk has the following parameter(s):

- *int calorie[n]:* the calorie counts for each cupcake

Returns

- *long:* the minimum miles necessary

Problem HackerRank Link:

https://www.hackerrank.com/challenges/marcs-cakewalk/problem

Github Submission Link:

https://github.com/Mdnadir311/DESIGN-ANALYSIS-AND-ALGORITHM/commit/6a99a0dbdb1000ca143ad565083e0991e4348ffb

Solution Steps:

• The problem is to **find the minimum number of miles Marc must walk after eating cupcakes with different calorie values**.

• Each cupcake requires Marc to walk $2^j \times c$**miles**, where **j** is the number of cupcakes already eaten and **c** is the calorie value.

• To minimize the total miles walked, the program **sorts the calorie values in descending order** so that cupcakes with higher calories are eaten first.

• The program then **loops through the sorted list of calories**.

• For each cupcake, it calculates the walking distance using the formula $2^i \times calorie[i]$.

• The calculated distance is **added to the total miles** Marc must walk.

• Finally, the program **returns and prints the minimum total miles** Marc needs to walk.

•

Complete JAVA Code:

```
import java.io.*;

import java.math.*;

import java.security.*;

import java.text.*;

import java.util.*;

import java.util.concurrent.*;

import java.util.regex.*;


class Result {


    public static long marcsCakewalk(List<Integer> calorie) {

        // Sort in descending order

        Collections.sort(calorie, Collections.reverseOrder());


        long miles = 0;


        for (int i = 0; i < calorie.size(); i++) {

            miles += (long)Math.pow(2, i) * calorie.get(i);

        }


        return miles;
```

```
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        String[] calorieTemp = bufferedReader.readLine().replaceAll("\\s+$", "").split("
");

        List<Integer> calorie = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            int calorieItem = Integer.parseInt(calorieTemp[i]);
            calorie.add(calorieItem);
        }

        long result = Result.marcsCakewalk(calorie);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

Observation/Solution:

Compiler Message

Success

Input (stdin)

```
3
1 3 2
```

Expected Output

```
11
```

*** DATE: 18th March 2026***